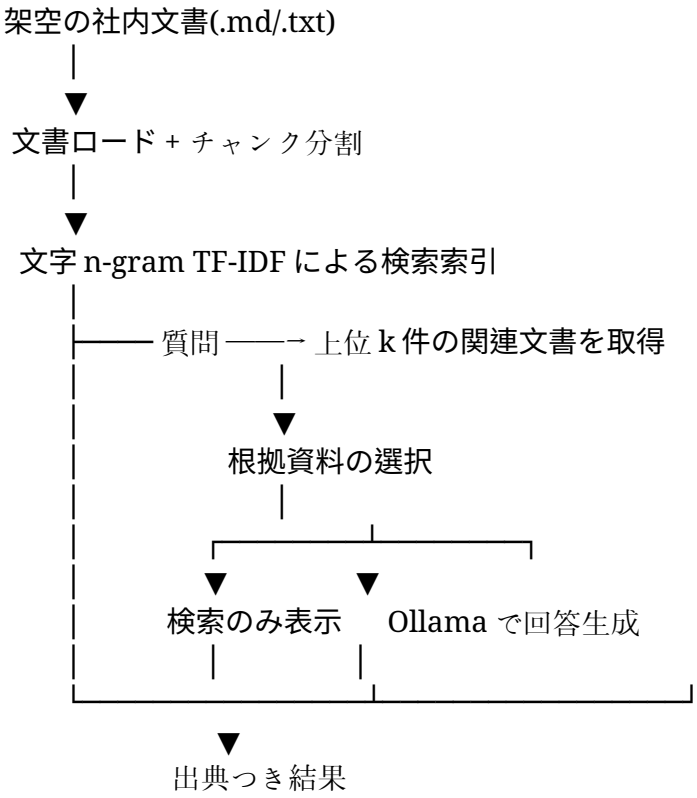
社内 RAG 開発シミュレーション｜PoC 実装ハンズオン

対象：架空企業の社内文書を用いた学習用 PoC。**実在する企業の非公開情報や個人情報を投入しない**こと。
目的：文書読み込み → 分割 → 検索 → LLM 回答 → 評価、という RAG の最小構成を再現する。
位置づけ：PoC 実装・評価フェーズ。本実装向けの認証・認可・監査ログ・アクセス権制御は対象外。

0. このハンズオンで作るもの

以下の流れを CLI で動作させます。



設計上の注意： この配布版は軽量なベースラインであり、検索には埋め込みモデル（Embedding）やベクトル DB ではなく、文字 n-gram TF-IDF を採用しています。日本語の単語分割に依存せず小さく動かすためです。既存の PoC 詳細設計で Embedding や Qdrant 等を選んでいる場合、これは**そのままの実装ではなく、検索方式を簡略化した教材**です。追加演習で交換します。

1. 学習目標・完了条件

- 文書を読み込み、チャンク分割がなぜ必要か説明できる
- 検索上位 3 件と類似度スコアを確認できる
- Ollama が利用できる環境では、根拠文書を与えて LLM 回答を生成できる
- 評価データから Recall@3 を再計算できる
- RAG の失敗例（見当違いの検索、根拠のない生成、データ中の悪意ある命令）を説明できる

前提環境

- Windows（PowerShell）または macOS / Linux のターミナル
- Python 3.10 以上推奨（本配布コードは Python 3 系の標準的な機能を使用）
- pip、ネット接続（初回パッケージインストールのみ）
- 任意：Ollama と利用可能な日本語対応モデル。サンプルコマンドは qwen2.5:7b を使います

セキュリティ前提： 外部公開サーバーにしない、実データを使わない、Ollama はローカルのまま利用する。Ollama の生成結果は安全性・正確性を保証しません。

2. 配布構成

```
rag_poc_handson/
├── HANDSON.md              # 本書
├── README.md               # 行コマンド早見表
├── requirements.txt        # Python 依存
├── main.py                 # CLI 入口
├── src/
│   ├── __init__.py
│   └── rag.py              # 分割・検索・回答の実装
├── data/
│   ├── documents/          # 架空文書 4 件
│   └── evaluation/
│       └── questions.json  # 検索評価用 8 問
├── tests/
│   └── test_rag.py         # 単体テスト 5 件
└── outputs/
    └── evaluation.json     # eval 実行後に生成
```

既存の RAG_Dev_Simulation に統合する場合は、既存フォルダを上書きせず、**まず poc_baseline/ など独立したサブフォルダ**に展開してください。以前の ZIP の実際のツリーを今回参照できないため、衝突しない形を推奨します。

3. STEP 1 | 環境構築

ZIP を展開して rag_poc_handson フォルダに移動します。

macOS / Linux

```
cd rag_poc_handson
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Windows PowerShell

```
cd rag_poc_handson
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

PowerShell の実行ポリシーで Activate.ps1 を実行できない場合でも、上記のように .venv\Scripts\python.exe を直接使えば進められます。以後の python は、macOS/Linux では有効化した環境の python、Windows では .venv\Scripts\python.exe に読み替えてください。

確認コマンド

```
python --version
python -m pip show scikit-learn
```

確認ポイント：Python と scikit-learn のバージョンが表示される。

4. STEP 2 | 架空データの確認

data/documents/ 内の 4 つの Markdown ファイルを開き、次の記述を探してください。

ファイル	テーマ	確認する情報
01_it_support.md	IT サポート	パスワード再設定、アカウントロック
02_remote_work.md	在宅勤務	申請期限、VPN
03_expense.md	経費精算	申請期限、承認者
04_incident.md	セキュリティ事故	報告先、記録項目

演習 2-A： 文章内の数字（15 分、17 時、5 営業日など）を正確に拾えるか、後の回答で確認します。

5. STEP 3 | 取り込み・チャンク分割

python main.py ingest

期待される形式：

```
{
 "documents": 4,
 "chunks": 4,
 "samples": ["01_it_support-001", "02_remote_work-001", "03_expense-001", "04_incident-001"]
}
```

src/rag.py の split_text() を読み、次を確認します。

- chunk_size=360：チャンクあたりの最大文字数
- overlap=60：隣り合うチャンクに重複させる文字数
- 文字数を増やした長い文書を追加すると、複数のチャンクに分割される

演習 3-A： data/documents/05_sample.md を作成し、架空の研修ルールを 500～700 文字書く。ingest を再実行し、documents と chunks の増加を確認する。

6. STEP 4 | 関連文書検索

python main.py ask "在宅勤務の申請期限はいつですか？"

出力の retrieved を確認します。

```
{
 "question": "在宅勤務の申請期限はいつですか？",
 "answer": "検索上位の文書を示します。回答生成には --ollama を指定してください。",
 "retrieved": [
  {"chunk_id": "02_remote_work-001", "source": "02_remote_work.md", "text": "...", "score": 0.0}
 ]
}
```

上記 score は**説明用のダミー**です。実際の値は検索コマンドの出力を見てください。検索上位には 02_remote_work.md が入るはずです。

仕組み： 各チャンクと質問を文字 2～4gram の TF-IDF ベクトルに変換し、コサイン類似度で順位付けしています。top_k=3 は類似度上位 3 チャンクを返す指定です。

演習 4-A： 次の 3 つで検索結果がどう変わるか観察する。

```
python main.py ask "経費申請の期限を知りたい" --top-k 1
python main.py ask "端末をなくしたときの連絡先は？" --top-k 3
python main.py ask "架空企業の社長の誕生日は？" --top-k 3
```

--min-score は検索1位のスコアが一定値未満なら回答を保留する簡易的な制御です。**スコアの高さは回答の事実性を保証しません**。

7. STEP 5 | Ollama で回答生成（任意）

Ollama をインストール済みでなければ、公式ドキュメントに従い導入してください。モデルのダウンロードには容量と通信量が必要です。使えるモデルは環境に合わせて変更できます。

```
ollama pull qwen2.5:7b
ollama list
```

Ollama が起動している状態で、以下を実行します。

```
python main.py ask "在宅勤務の申請期限はいつですか？" --ollama
```

期待する回答の意味は、**「前営業日の17時までに勤怠システムで申請し、所属長の承認を得る」**です。回答文面はモデルや実行環境によって異なります。引用元のファイル名・チャンクIDも回答に含まれているか確認してください。

任意のモデルを使う場合：

```
python main.py ask "経費精算の一次承認者は誰？" --ollama --model qwen2.5:7b
```

コードを読む場所：src/rag.py の ollama_generate()。取得チャンクをコンテキストとして渡し、「資料にないことは確認できないと回答する」指示を与えています。文書中の命令を信用しないよう指示していますが、**プロンプトだけで攻撃を完全防御することはできません**。

8. STEP 6 | 検索品質を自動評価

```
python main.py eval
```

評価結果は outputs/evaluation.json に保存されます。8問の質問に対して、期待文書が検索上位3件に含まれる割合を計算します。

Recall@3 = 期待文書が上位3件に含まれた質問数 / 評質問

配布時の初期データでは、実行確認環境で **8/8 = 1.00** でした。ただし文書が4件で候補を3件返す小さな評価なので、この数字だけでは高精度な検索と判断できません。評価は**検索段階のみ**で、LLMの回答品質を表しません。

演習 6-A：--top-k 1 で再評価し、Recall@1 と Recall@3 を比較する。

```
python main.py eval --top-k 1
```

演習 6-B：質問を言い換えて questions.json に追加し、検索性能の変化を観察する。

9. STEP 7 | 単体テスト

```
python -m unittest discover -s tests -v
```

確認ポイント：5件が OK になる。実行時の変更・環境によって失敗した場合は、テスト名とエラーを確認します。

テストの範囲：チャンク長・不正引数・文書読み込み・関連文書検索・高しきい値時の回答保留。

10. STEP 8 | 失敗を観察する演習

ここでは攻撃手順ではなく、**RAGがどのように誤るかを安全に評価**することに集中します。

試験	実施方法	観察点
未記載質問	「社長の誕生日は？」と質問	根拠不明でも検索候補が返る可能性

試験	実施方法	観察点
言い換え	「勤務場所の変更をいつ届ける？」と質問	表現差による検索漏れ
紛らわしい文書	似た申請期限の別の架空規程を追加	不正確な検索順位・回答の混同
文書追加	研修文書を追加して再評価	回答と索引がどのように変わるか
根拠不足	--min-score 1.1 を指定	原則として回答保留になること

注意：検索のみモードは回答本文を作りません。LLM 回答の評価は --ollama 実行後に人手で行います。

11. STEP 9 | 結果の記録

以下のテンプレートをコピーして、outputs/poc_report.md に記録します。

```
# PoC 評価メモ

## 実行環境
- OS / Python / scikit-learn / Ollama モデル:

## 取り込み結果
- 象文書:
- チャンク:

## 検索評価
- Recall@1:
- Recall@3:
- 代表的な成功例:
- 代表的な失敗例:

## LLM 回答の目視評価
- 回答は正しいか:
- 資料にないことを断定していないか:
- 出典が明示されているか:

## 本実装に向けた課題
- 埋め込みベクトル DB への移行:
- 文書アクセス権・認証認可:
- 引用根拠の照合:
- ログ・評価基盤:
- インジェクション耐性:
```

12. 展演習 | 本の PoC 設計へ張する

現在の構成を確認したあと、設計方針に合わせて次の順番で拡張します。

1. **Embedding 検索：** SearchIndex を SentenceTransformer 等の多言語埋め込みモデルに置き換え、質問とチャンクを意味ベクトルに変換する。モデルに応じたクエリ/文書プレフィックスを守る。
2. **ベクトル DB：** Qdrant または Chroma に文書 ID・本文・出典・ベクトルを保存する。取り込みの再実行で重複が発生しないよう ID を設計する。
3. **評価：** Recall@k に加え、MRR、引用の適合性、回答正確性、回答保留の妥当性を測定する。検索評価と生成評価を分離する。
4. **セキュリティ：** 認証・文書レベルの ACL、入力上限、取り込み検証、監査ログ、根拠文書の信頼境界を整理する。

この段階までは**本実装ではなく PoC** なので、全面的な UI 構築・認証基盤・クラウドデプロイは必須にしません。

13. トラブルシューティング

症状	確認すること
ModuleNotFoundError: sklearn	python -m pip install -r requirements.txt を実施した Python と、実行する Python が一致しているか
python が使えない	Windows は py、macOS は python3 で環境作成。仮想環境の Python 実行パスを使用
文字化け	ファイルを UTF-8 で保存し直す。Windows ターミナルでは UTF-8 表示を使用
Ollama 接続エラー	Ollama アプリ/サービスが起動しているか。ollama list にモデルがあるか
Ollama モデル未取得	ollama pull qwen2.5:7b を実施。使用モデル名を一致させる
回答が資料と違う	retrieved の本文・スコア・質問を確認し、検索の問題か生成の問題か切り分ける
新規文書が見つからない	data/documents 配下の .md / .txt か確認。索引は実行のたびに構築される

14. 完了チェックリスト

- □ Python 仮想環境を作成し依存を導入した
- □ ingest で文書件数とチャンク数を確認した
- □ ask で関連文書の上位順位を見た
- □ eval で Recall@1 と Recall@3 を比較した
- □ 単体テストが通った
- □ （任意）Ollama で回答し、元文書と照合した
- □ poc_report.md に課題と設計改善点を記録した

教材の限界

配布版は小規模な**比較用ベースライン**です。実際の企業内 RAG を本番運用するための構成ではありません。特に検索の類似度しきい値、LLM 出力の信用、権限に応じた検索範囲は未検証です。次フェーズで設計とのギャップをレビューしてください。